

FastBEMT Implementation

Davide Pusino

January 2026

1 Implementation Details

The efficiency of this solver stems from two primary optimizations:

- **Spatial Continuity:** We solve the root section, and then we use the converged ϕ value as an initial guess for the next section. Brent's method requires a bracket where a function sign change must happen. For each section after the first one, we initially set it at $[\phi_{prev} - \delta, \phi_{prev} + \delta]$ with $\delta = 1^\circ$, and expand the bracketed until the edges have different signs. This ensures tight bounds around the true ϕ which reduces the number of iterations required inside `scipy.optimize.root_scalar`.
- **Lazy Tabulation:** For every call to Neuralfoil, we first check if we already have the airfoil polars at that Re (rounded to the nearest 10000) and Ma (rounded to the nearest 0.1). If we do, we return C_l , C_d at the queried α for that Re and Ma regime. If we don't, we query Neuralfoil and save the polars at this new Re and Ma (rounded) in the cache.
- **Precomputing of Prandtl loss:** The Prandtl loss is precomputed on initialization of each Section object for 100 values of ϕ between 0.1° and 89.9° , and interpolated at evaluation time.

2 FastBEMT Algorithm

Algorithm 1 Complete FastBEMT Implementation with Sequential Bracketing

Require: Propeller geometry (r, c, θ) , Operating conditions (V_∞, Ω, ρ) **Ensure:** Total Thrust T , Total Torque Q

```
1: function FASTBEMT_SOLVER
2:   Initialize  $T \leftarrow 0, Q \leftarrow 0, \phi_{prev} \leftarrow \text{None}$ 
3:   Initialize empty dict Cache
4:   Precompute PrandtlLoss( $\phi$ ) for 100 values
5:   for each radial section  $i = 1$  to  $N$  do
6:     Create Section Object  $S_i$  with geometry  $(r_i, \Delta r_i, c_i, \theta_i)$ 
7:      $V_{local} \leftarrow \sqrt{V_\infty^2 + (\Omega r)^2}$ 
8:      $Re \leftarrow \frac{\rho V_{local} c}{\mu}$  ▷ Initial Reynolds guess
9:      $Ma \leftarrow \frac{V_{local}}{a_\infty}$  ▷ Initial Mach guess
10:     $\phi_i, dT, dQ \leftarrow \text{SOLVE}(\phi_{prev})$ 
11:     $T \leftarrow T + dT$ 
12:     $Q \leftarrow Q + dQ$ 
13:     $\phi_{prev} \leftarrow \phi_i$  ▷ Initial guess for next section
14:  end for
15:  return  $T, Q$ 
16: end function

17: function SOLVE( $\phi_{prev}$ )
18:  if  $\phi_{prev}$  exists then
19:     $\delta \leftarrow 1^\circ$ 
20:    while  $[\phi_{prev} - \delta, \phi_{prev} + \delta] \subset [0.1^\circ, 89.9^\circ]$  and  $\text{sign}(f(\phi_{prev} - \delta)) =$   

 $\text{sign}(f(\phi_{prev} + \delta))$  do
21:       $\delta \leftarrow 2\delta$ 
22:    end while
23:     $bracket \leftarrow [\phi_{prev} - \delta, \phi_{prev} + \delta]$ 
24:  else
25:     $bracket \leftarrow [0.1^\circ, 89.9^\circ]$ 
26:  end if
27:  Find  $\phi^*$  in  $bracket$  s.t.  $f(\phi^*) = 0$  using Brent's method
28:   $\alpha, a, a', dT, dQ \leftarrow \text{SECTIONPARAMETERS}(\phi^*, Re, Ma)$ 
29:  return  $\phi^*, dT, dQ$ 
30: end function
```

Algorithm 2 Calculation of $f(\phi)$

Require: ϕ, Re, Ma

```

1: function SECTIONPARAMETERS
2:    $\alpha \leftarrow \theta - \phi$   $\triangleright \alpha$  in degrees for Neuralfoil
3:    $Key \leftarrow \text{Round}(Re, 10^4), \text{Round}(Ma, 0.1)$ 
4:   if  $Key \notin \text{Cache}$  then
5:     Generate  $C_l, C_d$  polars via NeuralFoil and store in Cache
6:   end if
7:    $C_l, C_d \leftarrow \text{Interpolate}(\alpha, \text{Table}[Key])$ 
8:    $F \leftarrow \text{Interpolate}(\phi^*, \text{PrandtlLoss})$ 
9:    $C'_l \leftarrow C_l \cos \phi - C_d \sin \phi$ 
10:   $C'_d \leftarrow C_l \sin \phi + C_d \cos \phi$ 
11:   $a \leftarrow \frac{1}{\frac{4F \sin^2 \phi}{\sigma C'_l} - 1}$ 
12:   $a' \leftarrow \frac{1}{\frac{4F \sin \phi \cos \phi}{\sigma C'_d} + 1}$ 
13:   $v_a \leftarrow (1 + a)v_\infty$ 
14:   $v_t \leftarrow (1 - a')\Omega r$ 
15:   $W \leftarrow \sqrt{v_a^2 + v_t^2}$ 
16:   $Re \leftarrow \frac{\rho W c}{\mu}$ 
17:   $Ma \leftarrow \frac{W}{a_\infty}$ 
18:  return  $\frac{\sin \phi}{1+a} - \frac{V_\infty}{\Omega r} \left( \frac{\cos \phi}{1-a'} \right), Re, Ma$ 
19: end function

```
